TAC 359 Final Project Report

Ashley Zapata Minero

Sam Mui Tsan

Summary of The Results

Does Our Code Solve the Problem?

For the Taxi-v3 environment, we use a DQN agent to solve the problem and our model shows clear signs of learning and improving its performance. In most of the different training scenarios, we see a trend of increasing in the total reward and decreasing of the number of steps, meaning the taxi, in the late window, reaches the destination faster with fewer mistakes. The downward-sloping epsilon decay curve also shows that the agent moved gradually from exploration to exploitation over time. Although, we saw this performance throughout tests, the code we made does not solve the problem of taxi to passenger drop-offs. This is because we are working with a DQN, and as we see, in our trials adding other features such as replay buffer and saving it and reusing it during training is what actually helped it improved. Therefore, just by itself, it is harder, very unstable, and requires numerous modifications to use a DQN and find the optimal solution. That being said it is not unsolvable, if lots of optimizations are added, and a bunch of batches and optimal hyperparameterization is used, such as suggested in the outlook section it is solvable.

Hyperparameter Observations

In our experiment, the FAST MODE model performed poorly but it was used as a baseline model to do futher adjust each hyperparameter. From our observation, the number of episodes is the most impactful parameter, giving the most significant improvement in total rewards and the biggest reduction in the number of average steps. The second important hyperparameter is batch size. In contrast, the learning rate and replay buffer size did not meaningfully improve our base

line model performance. Also, lowering gamma unexpectedly resulted in a slight improvement. Ultimately, episode is the champion hyperparameter in improving model performance. That explains why the FULL MODE model, which has a large episode number and bigger batch size performs much better. Additionally, the replay_buffer, although constant, showed with a context inducer by saving to file, performed better than previous models and often would guess the correct drop-off. It would do so in a minimum of 5 steps, which showed improvement, even after only 200 episodes. Furthermore, this showed potential for greater than 200+ episodes and more fine-tuning.

General Take of RL on Taxi-v3

Reinforcement Learning is a good fit for Taxi-v3 because the agent doesn't rely on labeled data. It learns by interacting with the environment and receiving rewards or penalties. The agent learns Q-values, which estimate how good each action is, and updates them using the Bellman equation. With an epsilon-greedy strategy, it starts by exploring randomly and gradually shifts toward exploiting the actions with higher Q-values, improving its performance over time. So in practice, it is a good model to start with on the Taxi-v3 dataset.

Challenges

The biggest challenge when working with RL is the long training time. Because training takes so long, we have to use a smaller number of episodes, which limits how much the agent can learn. Thus, we cannot fully assess the effectiveness of hyperparameter tuning since the model may not have enough time to show the actual impact of each change. Furthermore, file compatibility is another issue we encountered where .h5 is not compatible with Tensorflow 2.15, so one of us had to implement a SavedModel approach using files. So being up to date with the TensorFlow compatibility files is essential.

Concepts it Compares To (model/techniques)

Reinforcement Learning connects to several concepts we have learned, but in different ways than supervised or unsupervised models. Differently to autoencoders or GANs, which focus on reconstructing data or generating new samples, RL focuses on decision-making through interaction with an environment. Instead of a discriminator, the RL agent receives rewards that guide its learning. Also, instead of optimizing reconstruction loss, it optimizes the long-term expected return.

One meaningful comparison is to Markov models and sequential modeling techniques. Taxi-v3 follows the Markov Decision Process (MDP) framework, where each state transition depends only on the current state and action. Our implemented replay buffer, stored in a JSON file, does not create a Markov chain; instead, it stores past state transitions. This allows the agent to learn from them more efficiently. When we reused previous experiences across training runs (e.g., 300 episodes reused before training the next 300), the agent benefited from having a larger and more diverse transition history.

Finally, similar to other models we studied, RL requires significant fine-tuning and additional features such as replay buffers, epsilon decay, and batch updates to perform well. The professor's approach introduces the fundamental components, and our approach of adding some additional features, such as an extended replay buffer shows that additional optimizations are essential for an effective DQN agent.

Approach

Our approach began with running both the FULL and FAST versions of the Reinforcement Learning model. As expected, the initial runs showed different results between the two. However, it wasn't until the third combination that we started noticing limitations in our Reinforcement Learning setup.

At first, we assumed that the .h5 model was saving its learning and reusing that information. We later learned that this was not the case. In reality, saving progress would require writing the learned data to a file and then integrating it back into the model so it has proper context. Understanding this allowed us to reinterpret our earlier findings and better understand the foundations of Reinforcement Learning.

Another interesting part of our approach involved exploring policies. We initially tried applying a custom policy, but we learned that policies do not help much unless our goal is to analyze what the RL model has already learned and preserve that behavior for comparison. Through this project, we realized that the policy is simply the model's way of interpreting the environment, and it develops its own policies through training.

Overall, our approach was very data-driven. We often focused on adjusting one parameter at a time to clearly see the effect it had on the model's performance. Then we later applied the combinations to see it in full action. As we did each step, we gave our insights on what we believed was happening.

Hyperparameter Tuning Findings

Initial Run Using Default Hyperparameters

- **Original Fast Mode Model**

  Table 1 (1st row)/ https://drive.google.com/file/d/19t53xqyw_fdgATby4i5ZZ9YKP2b4-t_u/view?usp=sharing


```
"mode": "FAST",
"episodes": 200,
"max_steps": 100,
"batch_size": 32,
"gamma": 0.99,
"epsilon_start": 1.0,
"epsilon_min": 0.05,
"epsilon_decay": 0.995,
"replay_buffer_size": 2000,
"target_update_freq": 10,
"learning_rate": 0.001,
```


  As this instance of running the model was operating in Fast mode. The model can run a maximum of 100 steps every episode. The model seems to learn something after training, as the reward increased and the average number of steps decreased toward the later window. However, the reward is still negative when the model is complete, showing that the learning is not significant yet.

- **Full Mode Model**: training time from 6:55pm to 11:14pm

  Table 1 (2nd row)/ https://drive.google.com/file/d/1PuCNLbJ6344b9XL6LYCTIRvIx6NwhBKf/view?usp=drive_link


```
"mode": "FULL",
"episodes": 1000,
"max_steps": 200,
"batch_size": 64,
"gamma": 0.99,
"epsilon_start": 1.0,
"epsilon_min": 0.01,
"epsilon_decay": 0.995,
"replay_buffer_size": 10000,
"target_update_freq": 10,
"learning_rate": 0.0005
```


  Now we run the model at full mode thoroughly. The rewards are quite volatile in the first 400 episodes as the agent still explores the environment and has not yet learn the effective policy. But it gets more stable after that, demonstrating that the agent has begun to exploit what it has learned and the rewards have converged toward an optimal strategy. The agent starts by choosing actions that are very random (epsilon=1). The exploration rate decays

steadily, which means that the agent increases exploitation of the learned q-value. From the 600th episode and on, the epsilon becomes very small, indicating that the agent relies mostly on its learned policy. Total reward gained was significantly high (+459.06) and the number of steps also decreased significantly (decreased by +145.73 steps) between the early and late window. This is a well performance model.

In full mode, the agent learn gradually, while in fast mode, the agent barely learns, so the step numbers stay high. It takes a long time to finish the episode (wandering around, no strategy).

However, we still cannot tell which hyperparameter is mainly responsible for the improvement. Hence, we decided to tune five different hyperparameters separately to see which one has the strongest impact. We use Fast Mode as our baseline model. And, to make the comparison fair, we only change one hyperparameter at a time while keeping all the others the same as the original model. This helps us isolate the effect of each hyperparameter more effectively.

Individual Hyperparameter Tuning

- **Batch size tuning model**

  Table 1 (3nd row)/ https://drive.google.com/file/d/1DFwt0kj0V7y8OLfoenA5_yzSze_s4cVR/view?usp=drive_link

```
"batch_size": 64, #Change from 32 to 64
```

  Increasing the batch size to 64 makes training much slower than with 32. The reward improved but moderately, showing that there is some learning, but it is not strong enough. Steps also decreased toward the end (+7.80), meaning the agent got slightly more efficient than the original fast mode (+4.258). However, the reward in the late window is still negative, so tuning batch size yields some efficiency but not a lot.

- **Learning rate tuning model**

  Table 1 (4rd row)/ https://drive.google.com/file/d/1ylflYcXDvUTyxqDkG6kQdf4XSFWsvNWP/view?usp=drive_link

```
"learning_rate": 0.0005, # change from 0.001 to 0.005
```

Tuning the learning rate to 0.0005 somehow worsens the performance of the model. It gains lower rewards compared to the original Fast mode model (+170.48 vs. +154.10). The steps also did not decrease significantly, just decreased by +0.50, indicating that the model did not learn well. So, adjusting the learning rate alone might not give a better performance.

- **Episode tuning model**

Table 1 (5th row)/ https://drive.google.com/file/d/1Vr_EF5e5ah9FTRdhl3ZR1x_MbpihpaiB/view?usp=drive_link

```
"episodes": 400, #change from 200
```

Increasing the number of episodes shows a significant reward improvement, compared to the original Fast Mode model (+240.64 vs. +170.48). The step numbers also decreased significantly in the late window, decreased by +24.94 steps, indicating that increasing number of episodes is the right approach to improve the performance of the model.

- **Gamma tuning model**

Table 1 (6th row)/ https://drive.google.com/file/d/1Z5Rf9_NXk_tkYhPF0hcX96LAGUl0XRnu/view?usp=drive_link

```
"gamma": 0.95, #change from 0.99 to 0.95
```

Gamma decides how much the model cares about future rewards. If the gamma is high, the agent focuses on long-term outcomes and vice versa. However, this model with a lower gamma (from 0.99 to 0.95) yields a slight improvement over the original Fast mode model, telling us that lowering gamma might help the model perform better. But the difference is not significant. We also need to take into account the models' uncertainty.

- **Replay Buffer Size tuning model**

Table 1 (7th row)/
https://drive.google.com/file/d/1HojuL4Y9NM3RkGp_cMNLZmMLzO_4p49i/view?usp=drive_link

```
"replay_buffer_size": 5000, #change from 2000 to 50000
```

Increasing the replay buffer size allows the model to store more past steps and learn from them. However, the model did not seem to perform better than the original model. So, we came to the conclusion that tuning the replay buffer size alone does not make a big difference.

Take away: We found that the number of episodes is the most important hyperparameter for improving model performance. Batch size also shows a clear and meaningful improvement. We also discovered some interesting results: increasing the learning rate and expanding the replay buffer are expected to help, but in our case, they did not improve the model. Meanwhile, lowering gamma should have worsened performance, but it actually gave a slight improvement. However, because we used a small number of episodes for these tuning runs (to save training time), we are not fully sure how reliable these results are. What we can say for certain is that the episode count is the most impactful hyperparameter overall.

For more exploration, we also try to tune multiple hyperparameters randomly to see how it will affect the model performance. We came up with the follwoing three combinations of hyperparameters:

Testing Selected Hyperparams in Combinations

- **Hyperparam tuning combination 1**

Table 1 (8th row)/ https://drive.google.com/file/d/1hf1zT3moD0nNWBWrud8N628e9Bj5c-n9/view?usp=drive_link

```
"mode": "FAST",
"episodes": 300,
"max_steps": 100,
"batch_size": 64,
"gamma": 0.95,
"epsilon_start": 1.0,
"epsilon_min": 0.05,
"epsilon_decay": 0.990,
"replay_buffer_size": 2000,
"target_update_freq": 10,
"learning_rate": 0.002,
```

This run was interesting because, for the total rewards per episode, it achieved a reward of -55 during the 168th episode. It is still relatively low, but in comparison to the original, the learning seems to have improved drastically. Especially the rewards, although they still range in the negatives. This is evident because the number of episodes was 300; more

episodes could prove beneficial with this set of parameters. Additionally, the average loss per episode was extremely low, with the highest value being approximately 0.189. Furthermore, for the Steps per Episode, it was observed that after 146 epochs, the number of steps reached 35. It was interesting to see that after 146 epochs, the number of steps remained consistent. Perhaps it had gained insight into the data.

- **Hyperparam tuning combination 2**

Table 1 (9th row)/ https://drive.google.com/file/d/1h8IRqdYcIol39t4EYn8zHp2SNIPlf23O/view?usp=drive_link

```
"mode": "FAST",
"episodes": 200,
"max_steps": 50,
"batch_size": 20,
"gamma": 0.95,
"epsilon_start": 1.0,
"epsilon_min": 0.05,
"epsilon_decay": 0.985,
"replay_buffer_size": 2000,
"target_update_freq": 10,
"learning_rate": 0.001,
```

This run's highest total reward, seen in the Total Reward per Episode chart, is -59. In this particular run, I changed the number of steps from 100 to 50 to see if it would make any difference. I also used a learning rate of 0.001, which is known to be one that is good learning rate in machine learning. And used a batch size of 20. The shape of the graph indicates that this type of parameterization introduces significant noise that the agent must navigate through. Also, the epsilon exploration rate seems to be good for this run. Additionally, I found it peculiar that there was mostly no training loss, and even the steps per epoch remained consistent. Perhaps the data the agent got was very consistent, and perhaps since it was in smaller batches way easier to understand the underlying pattern of the data, so the agent was repeating the same behavior.

- **Hyperparam tuning combination 3**

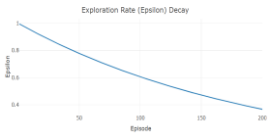
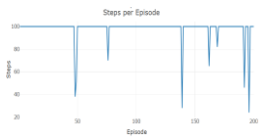
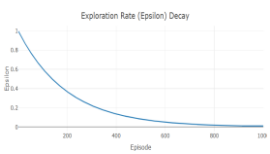
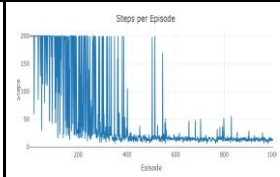
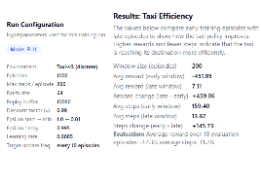
Table 1 (10th row)/
https://drive.google.com/file/d/1H1HlZFNSLZR5Bq_eH484T9alU5xsMKUE/view?usp=drive_link

```
"mode": "FAST",
```

```
"episodes": 200,
"max_steps": 200,
"batch_size": 32,
"gamma": 0.95,
"epsilon_start": 1.0,
"epsilon_min": 0.05,
"epsilon_decay": 0.985,
"replay_buffer_size": 2000,
"target_update_freq": 10,
"learning_rate": 0.001,
```

We explored another combination of hyperparameter adjustments and observed an interesting result. The graphs show that epsilon unexpectedly resets to 1.0 around episodes 200 and 400, which we don't know why. These resets indicate that the agent's training process was restarted multiple times, which also explains the sharp drops in total reward at the same points. Because the agent began exploring randomly again after each reset, the learning process was repeatedly interrupted rather than building on long-term experience from the replay buffer. The steps-per-episode graph also reflects this pattern. In the late window, the number of steps becomes volatile, indicating that the agent re-entered a highly exploratory phase and had not yet relearned an efficient policy.

Findings

	Total Reward per Episode	Exploration Rate Decay	Average Training Loss per Episode	Steps per Episode	Configuration & Efficency
1					
2					

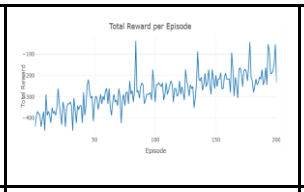
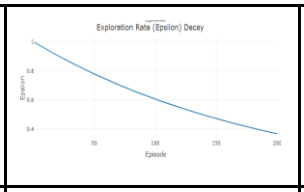
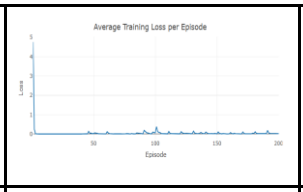
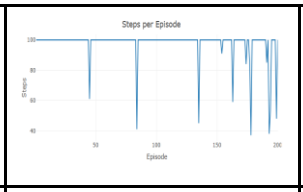
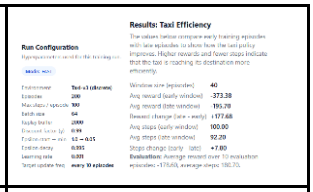
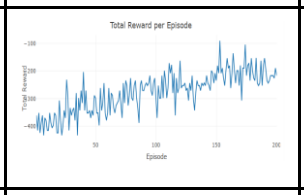
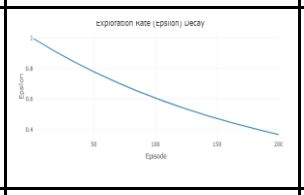
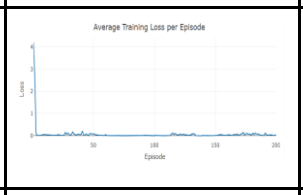
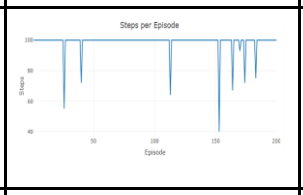
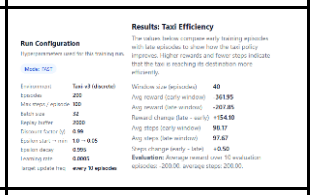
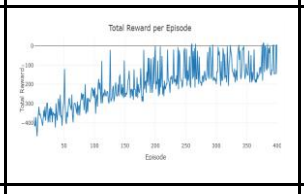
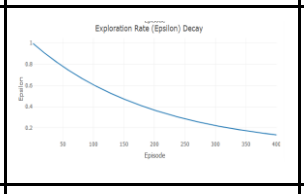
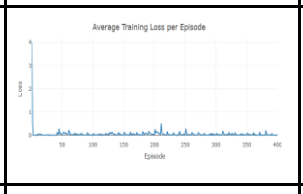
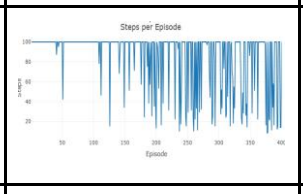
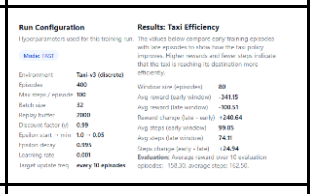
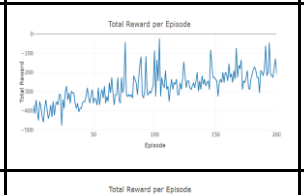
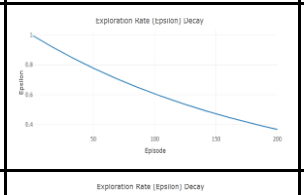
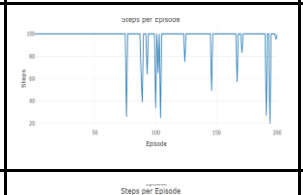
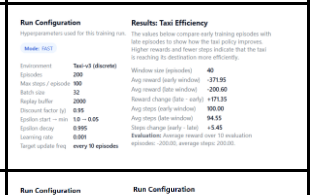
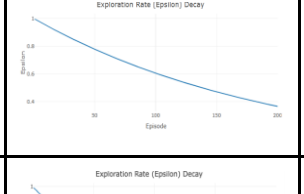
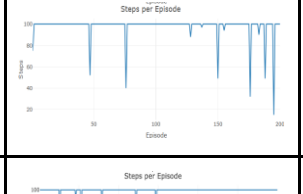
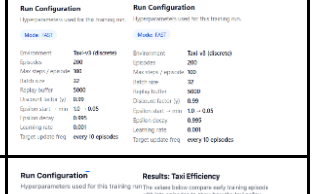
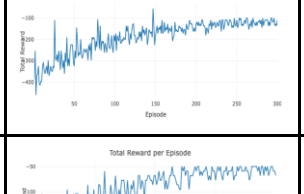
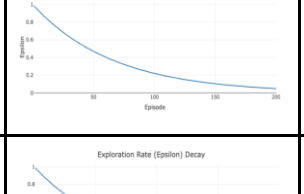
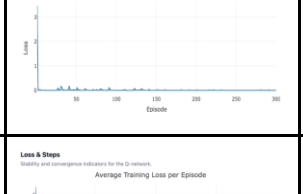
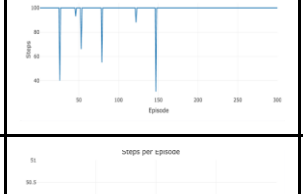
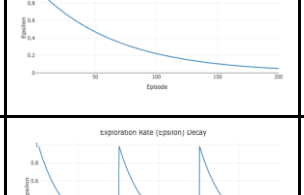
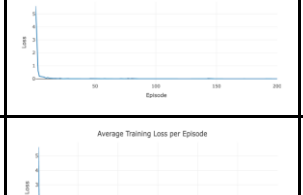
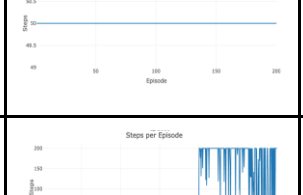
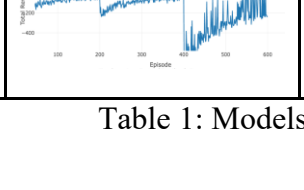
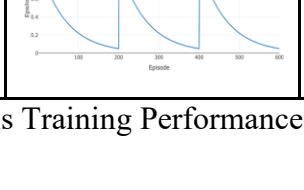
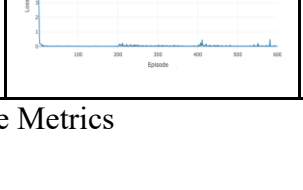
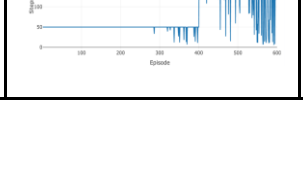
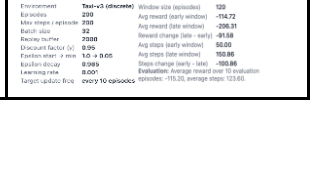
3						
4						
5						
6						
7						
8						
9						
10						

Table 1: Models Training Performance Metrics

	Window size	Avg Reward (early window)	Avg Reward (late window)	Reward change (late-early)	Avg steps (early window)	Avg steps (late window)	Steps change (early - late)

Fast mode	40	-369.55	-199.07	+170.48	100.00	95.42	+4.58
Full mode	200	-451.95	7.11	+459.06	159.40	13.67	+145.73
Batch size tuning	40	-373.38	-195.70	+177.68	100.00	92.20	+7.80
LR tuning	40	-361.95	-207.85	+154.10	98.17	97.67	+0.50
Episode tuning	80	-341.15	-100.51	+240.64	99.05	74.11	+24.94
Gamma tuning	40	-371.95	-200.60	+171.35	100.00	94.55	+5.45
Replay buffer tuning	40	-360.52	-205.50	+155.02	99.38	94.65	+4.72
Combination 1	60	-304.27	-121.15	+183.32	98.32	100.00	-1.68
Combination 2	40	-153.50	-57.65	+95.85	50.00	50.00	+0.00
Combination 3	120	-114.72	-206.31	-91.58	50.00	150.86	-100.86

Table 2: Reward and Step Improvements Across DQN Training

Outlook / Future enhancements, risks

Outlook

- When the model starts with zero context, it tends to overfit and does not learn efficiently. It performs much better when it has actual context and can use past patterns to reinforce what it knows. This is why, even though the model improved, many runs stayed in negative reward ranges (around -100 to -50). This shows how essential the replay buffer is in reinforcement learning. This is because the buffer holds the context and previous decisions that the model uses to improve.
- The most successful iteration we observed came from the 1000-batch configuration. This suggests that even larger batch sizes(similar to big-data real-world applications) can be very effective, as long as the model is focused on a consistent type of data. With 1000+ batches, the model was better able to generalize. A key factor here is saving the buffer to a file and reusing it in future training iterations.
- Through the combination tests, we also observed that a learning rate of 0.0001 performed consistently well. Epsilon also had a strong impact on results; for example, $\varepsilon = 0.995$ worked very effectively, though different combinations naturally produce different behaviors.
- The model also appeared to be training on noise, similar to an autoencoder trained on noisy datasets, but in a reinforcement learning context. Since the taxi environment has roughly 500 discrete states, the model explores a large number of possible scenarios. When we reused the buffer from a previous run, the second iteration produced surprisingly strong

results, with many episodes finishing in 5, 8, or 11 steps, showing improved generalization and adaptation to the environment.

Future enhancements

- One interesting future enhancement would be to explore the idea the professor mentioned: having one agent train another agent, effectively using a teacher–student RL setup.
- Another possibility is to have a secondary agent whose goal is to track and evaluate hyperparameter effects. Every time it sees new behavior, it updates its knowledge. Over time, this could allow the system to automatically search for near-optimal hyperparameters since the exploration cost is low for an agent.

Risks

- In the Taxi-v3 environment, the model can still make mistakes such as incorrect drop-offs. Even with enough training to reach 80% accuracy, a real-world system would require human monitoring and regular updates to the model's policies.
- There is also the risk of poor performance if someone chooses ineffective hyperparameters. Incorrect settings can significantly reduce learning efficiency. This is why having an experienced practitioner or the automated parameter-search idea mentioned earlier would be beneficial.

References, incl. AI usage statement

- Used AI to help debug with dqn_taxi_model.h5, because it was not running. This was due to an extra parameter that the TensorFlow version I was using did not recognize. I discovered through AI that the issue was due to the Keras version I was using, 2.15, which no longer supported .h5 files and instead utilized TensorFlow. This helped resolve the issue by converting the .h5 file to a standard folder-type output of results.
- I also wanted to explore policies, so I learned through AI how to effectively implement them and how they impact the flow of RL models. For instance, I believed that perhaps it would be beneficial if the RL model could have, say, 10% of its knowledge derived from policies and 90% of its knowledge gained through exploration. Hence, I was attempting that, but I think perhaps for this project it would be better to see how the model itself behaves before considering the potential bias the RL model might have towards that 10%.
- I also used AI to help understand how to keep the buffer running and let the agent continue using that to improve its learning. It was very interesting to see how much it improved as a result.
- Approximate percentage of code written by AI: 20%